

2주차 예습과제_김유진

2주차 예습과제

[2장] 사이킷런으로 시작하는 머신러닝

- 사이킷런 소개와 특징
 - 첫 번째 머신러닝 만들어보기 - 붓꽃 품종 예측하기
 - 사이킷런의 기반 프레임워크 익히기
 - Model Selection 모듈 소개
 - 데이터 전처리
 - 사이킷런으로 수행하는 타이타닉 생존자 예측
-

01. 사이킷런 소개와 특징

- 파이썬 머신러닝 라이브러리 중 가장 많이 사용되는 라이브러리
- 머신러닝을 위한 매우 다양한 알고리즘과 개발을 위한 편리한 프레임워크와 API 제공

```
pip install scikit-learn== 1. 0. 2
import sklearn
print(sklearn.__version__)
```

02. 첫 번째 머신러닝 만들어 보기 - 붓꽃 품종 예측하기

- 붓꽃 데이터 세트는 꽃잎의 길이와 너비, 꽃받침의 길이와 너비 피처를 기반으로 꽃의 품종을 예측

- 분류는 대표적인 지도학습 방법 중 하나이며, 지도학습이란 명확한 답이 주어진 데이터를 먼저 학습한 뒤 미지의 정답을 예측하는 방식
- 사이킷런 패키지 내의 모듈명: sklearn 으로 시작
- 붓꽃 데이터 세트를 생성하는 데에는 load_iris()를 이용하며, ML 알고리즘은 의사 결정 트리 알고리즘으로, 이를 구현한 DecisionTreeClassifier를 적용함

주요 모듈 및 클래스

sklearn.datasets	사이킷런 자체적 제공 데이터 세트 생성 모듈	load_iris()
sklearn.tree	트리 기반 ML 알고리즘 구현 클래스 모임	DecisionTreeClassifier
sklearn.model_selection	학습/테스트 데이터 분리 및 파라미터 튜닝 모듈	train_test_split()

예측 프로세스

1. **데이터 세트 분리**: 데이터를 학습 데이터와 테스트 데이터로 분리
2. **모델 학습**: 학습 데이터를 기반으로 ML 알고리즘을 적용해 모델을 학습시킴
3. **예측 수행**: 학습된 ML 모델을 이용해 테스트 데이터의 분류(붓꽃 종류)를 예측
4. **평가**: 예측된 결과값과 테스트 데이터의 실제 결과값을 비교해 ML 모델 성능 평가

03 사이킷런의 기반 프레임워크 익히기

Estimator 이해 및 fit(), predict() 메서드

- 사이킷런은 ML 모델 학습을 위해서 fit()을, 학습된 모델의 예측을 위해 predict() 메서드를 제공
- **Classifier** : 분류 알고리즘을 구현한 클래스
- **Regressor**: 회귀 알고리즘을 구현한 클래스
- **Estimator** = Classifier와 Regressor를 합쳐서 부르는 것

사이킷런의 주요 모듈

분류	모듈명	설명
예제 데이터	sklearn.datasets	사이킷런에 내장된 예제 데이터 세트
피처 처리	sklearn.preprocessing	데이터 전처리 기능
피처 처리	sklearn.feature_selection	중요 피처 선택 기능
피처 처리	sklearn.feature_extraction	텍스트/이미지 데이터 벡터화
피처 처리 & 차원 축소	sklearn.decomposition	차원 축소 알고리즘
데이터 분리, 검증 & 파라미터 튜닝	sklearn.model_selection	데이터 분리, 교차검증, GridSearch
평가	sklearn.metrics	분류, 회귀, 클러스터링 성능 평가
ML 알고리즘	sklearn.ensemble	랜덤포레스트, AdaBoost, Gradient Boost
ML 알고리즘	sklearn.linear_model	선형회귀, Ridge, Lasso, 로지스틱 회귀
ML 알고리즘	sklearn.naive_bayes	나이브 베이즈
ML 알고리즘	sklearn.neighbors	K-NN
ML 알고리즘	sklearn.svm	SVM
ML 알고리즘	sklearn.tree	의사결정트리
ML 알고리즘	sklearn.cluster	K-means, 계층형, DBSCAN
유틸리티	sklearn.pipeline	피처 처리 등 변환, ML 알고리즘 학습

내장된 예제 데이터 세트

사이킷런에 내장 되어있는 데이터 세트는 분류나 회귀를 연습하기 위한 **예제용도의 데이터 세트**와 분류나 클러스터링을 위해 **표본 데이터로 생성될 수 있는 데이터 세트**로 나뉘어진다.

API 명	설명
datasets.load_boston()	회귀 용도이며, 미국 보스턴의 집 피처들과 가격에 대한 데이터 세트
datasets.load_breast_cancer()	분류 용도이며, 위스콘신 유방암 피처들과 악성/양성 레이블 데이터 세트
datasets.load_diabetes()	회귀 용도이며, 당뇨 데이터 세트

API 명	설명
<code>datasets.load_digits()</code>	분류 용도이며, 0에서 9까지 숫자의 이미지 픽셀 데이터 세트
<code>datasets.load_iris()</code>	분류 용도이며, 붓꽃에 대한 피처를 가진 데이터 세트
<code>datasets.make_classifications()</code>	분류를 위한 데이터 세트 생성.
<code>datasets.make_blobs()</code>	클러스터링을 위한 데이터 세트 무작위 생성

- 데이터 세트는 일반적으로 **딕셔너리 형태**
- 데이터 키는 피처들의 데이터 값을 가리킴
- **data**는 피처의 데이터 세트를 가리킴
- **target**은 분류 시 레이블 값, 회귀일 때는 숫자 결괏값 데이터 세트
- **target_names**는 개별 레이블의 이름을 나타냄
- **feature_names**는 피처의 이름을 나타냄
- **DESCR**은 데이터 세트에 대한 설명과 각 피처의 설명을 나타냄

04 Model Selection 모듈 소개

학습/테스트 데이터 세트 분리 - `train_test_split()`

원본 데이터 세트에서 학습/테스트 데이터 세트 분리

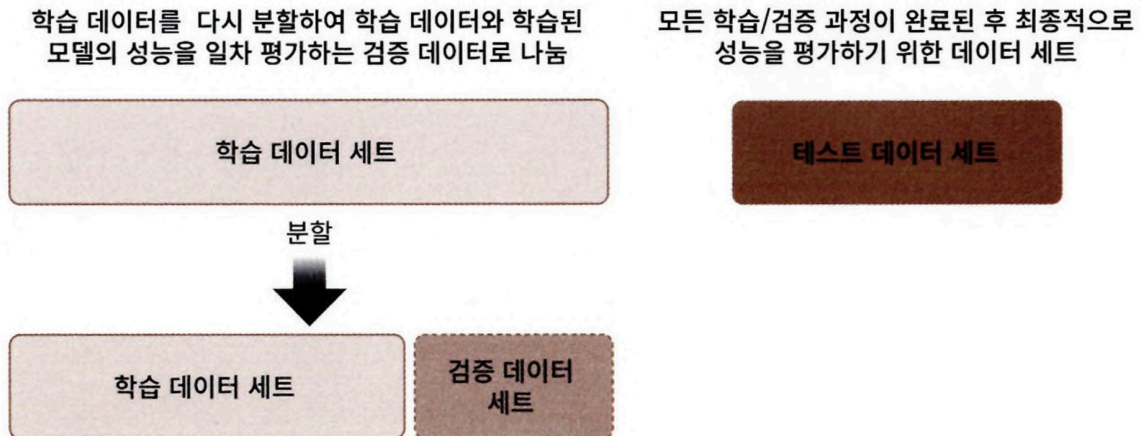
- **test_size**: 전체 데이터에서 테스트 데이터 세트 크기를 얼마로 샘플링할 것인가를 결정함. default : 0.25
- **train_size**: 전체 데이터에서 학습용 데이터 세트 크기를 얼마로 샘플링할 것인가를 결정함
- **shuffle**: 데이터를 분리하기 전에 데이터를 미리 섞을지 결정함. default: True
- **random_state**: `random_state`는 호출할 때마다 동일한 학습/테스트용 데이터 세트를 생성하기 위해 주어지는 난수 값

교차 검증

- 과적합 (Overfitting) : 모델이 학습 데이터에만 과도하게 최적화된 상태 → 새로운 데이터일 때 성능 떨어짐
- 교차 검증: 여러 개의 학습/ 검증 세트를 이용하여 반복 평가 (대부분의 ML 모델 평가 과정)

1차 검증 → 교차 검증

최종 평가 → 테스트 데이터



K 폴드 교차 검증

K개의 데이터 폴드 세트를 만들어서 K번만큼 각 폴드 세트에 학습과 검증 평가를 반복적으로 수행하는 방법

사이킷런 클래스: `KFold`, `StratifiedKFold`



Stratified K 폴드

- 불균형한 분포도를 가진 레이블 (결정 클래스) 데이터 집합을 위한 K 폴드 방식

↳ 특정 레이블 값이 특이하게 많거나 매우 적어서 값의 분포가 한쪽으로 치우치는 것

- K 폴드가 레이블 데이터 집합이 원본 데이터 집합의 레이블 분포를 학습 및 테스트 세트에 제대로 분배하지 못하는 경우의 문제 해결
- 원본 데이터의 레이블 분포도 특성을 반영한 학습 및 검증 데이터 세트를 만들 수 있으므로 왜곡된 레이블 데이터 세트에서는 반드시 Stratified K 폴드를 이용해 교차 검증

cross_val_score()

교차 검증을 간단하게 수행하는 API

```
cross_val_score(estimator, X, y=None, scoring=None, cv=None, n_jobs=1)
```

GridSearchCV

교차 검증 기반으로 하이퍼 파라미터의 최적 값을 순차적으로 적용 및 도출

- **estimator**: 사용할 알고리즘 객체
- **param_grid**: 튜닝을 위해 사용할 파라미터명과 값을 지정한 딕셔너리
- **scoring**: 예측 성능을 측정할 평가 방법
- **cv**: 교차 검증 분할 세트 개수
- **refit**: 디폴트 (True)로 생성 시 가장 최적의 파라미터로 모델 재학습

05 데이터 전처리

데이터 인코딩

- 머신러닝을 위한 대표적인 인코딩 방식은 레이블 인코딩과 원-핫 인코딩이 있음

레이블 인코딩

- 카테고리 피처를 코드형 숫자 값으로 변환하는 것
- `LabelEncoder` 클래스로 구현
- `LabelEncoder` 를 객체로 생성한 후 `fit()` 과 `transform()` 을 호출해 레이블 인코딩 수행
- 문자열 값을 **숫자형** 카테고리 값으로 변환

원-핫 인코딩

- 피처 값의 유형에 따라 새로운 피처를 추가해 고유값에 해당하는 칼럼에만 1을 표시하고, 나머지 칼럼에는 0을 표시하는 방식
- `OneHotEncoder` 클래스 사용
- 주의할 점 - 입력 값으로 2차원 데이터 필요

- 변환 값이 희소 행렬 형태이므로 다시 `toarray()` 메서드를 이용해 밀집 행렬로 변환해야함

피처 스케일링과 정규화

- 피처 스케일링: 서로 다른 변수의 값 범위를 일정한 수준으로 맞추는 작업

표준화(Standardization)

- 데이터의 피처 각각이 평균이 0이고 분산이 1인 가우시안 정규분포를 가진 값으로 변환하는 것
- 서로 다른 피처의 크기를 통일하기 위해 크기를 변환해주는 개념

$$x_{i_new} = \frac{x_i - \text{mean}(x)}{\text{stdev}(x)}$$

StandardScaler

- 표준화를 쉽게 지원하기 위한 클래스 → 개별 피처를 평균이 0, 분산이 1인 값으로 변환

MinMaxScaler

- 데이터값을 0과 1 사이의 범위값으로 변환 (음수 값은 절댓값으로 변환)

학습 데이터와 테스트 데이터의 스케일링 변환 시 유의점

학습 데이터와 테스트 데이터 스케일링 시 주의

1. 학습 데이터로 `fit()` 수행
2. 테스트 데이터는 `transform()`만 수행

(테스트 데이터로 다시 새로운 스케일링 기준 정보를 만들게 되면 학습 데이터와 테스트 데이터의 스케일링 기준 정보가 달라지기 때문에 올바른 예측 결과 도출 X)

[3장] 평가

- 정확도
- 오차 행렬
- 정밀도와 재현율
- F1 스코어
- ROC 곡선과 AUC

01 정확도

- 정확도 = $\frac{\text{예측 결과가 동일한 데이터 건수}}{\text{전체 예측 데이터 건수}}$

→ 직관적으로 모델 예측 성능으로 나타내는 평가 지표

- 한계 : 이진 분류의 경우 데이터 구성에 따라 모델 성능 왜곡 가능

불균형한 레이블 값 분포에서 모델 성능 판단할 때 부적합

02 오차 행렬

학습된 분류 모델이 예측을 수행하면서 얼마나 헛갈리고 있는지도 함께 보여주는 지표

⇒ 이진 분류의 예측 오류가 얼마인지 + 어떠한 유형의 예측 오류가 발생하고 있는지 나타냄

실제/ 예측	Negative	Positive
Negative	TN	FP
Positive	FN	TP

- TN는 예측값을 Negative 값 0으로 예측했고 실제 값 역시 Negative 값 0
- FP는 예측값을 Positive 값 1으로 예측했는데 실제 값은 Negative 값 0
- FN 예측값을 Negative 값 0으로 예측했는데 실제 값은 Positive 값 1
- TP 예측값을 Positive 값 1으로 예측했고 실제 값 역시 Positive 값 1

정확도 = 예측 결과와 실제 값이 동일한 건수 / 전체 데이터 수 = $(TN + TP) / (TN + FP + FN + TP)$

03 정밀도와 재현율

- 정밀도 = $TP / (FP + TP)$

→ Positive로 예측한 대상 중 실제로 Positive인 비율

- 재현율 = $TP / (FN + TP)$

→ 실제 Positive 중 모델이 일치하게 예측한 비율

- 재현율이 상대적으로 더 중요한 경우: 실제 Positive 양성인 데이터 예측을 Negative로 잘못 판단하게 되면 업무상 큰 영향이 발생하는 경우
- 정밀도가 상대적으로 더 중요한 경우: 실제 Negative 음성인 데이터 예측을 Positive로 잘못 판단하게 되면 업무상 큰 영향이 발생하는 경우

정밀도/재현율 트레이드 오프

- 어느 한 쪽을 강제로 높이면 다른쪽 수치는 떨어지기 쉬운 상호 보완적 평가 지표
- 예측 확률이 큰 레이블 값으로 예측
- 임계값을 낮추면 **재현율 증가, 정밀도 감소**

정밀도와 재현율의 맹점

- **정밀도가 100% 되는 방법:** 확실한 기준이 되는 경우만 Positive로 예측하고 나머지는 모두 Negative로 예측
- **재현율이 100% 되는 방법:** 모두 Positive로 예측

04 F1 스코어

$$F1 = \frac{2}{\frac{1}{recall} + \frac{1}{precision}} = 2 * \frac{precision * recall}{precision + recall}$$

05 ROC 곡선과 AUC

ROC 곡선

- 수신자 판단 곡선
- 머신러닝의 이진 분류 모델의 예측 성능을 판단하는 중요한 지표
- **FPR(False Postivie Rate)이 변할 때 TPR(True Positive Rate)이 어떻게 변하는지를 나타내는 곡선**
- X축: FPR, Y축: TPR

$$FPR = FR / (FP + TN) = 1 - TNR = 1 - 특이성$$

- **민감도(TPR)**은 실제값 Positive가 정확히 예측되어야 하는 수준을 나타냄
- **특이성(TNR)**은 실제값 Negative가 정확히 예측되어야 하는 수준을 나타냄

AUC

- ROC 곡선 밑의 면적을 구한 값
- 1에 가까울수록 좋은 수치
- `roc_auc_score()`